Deep Learning

Multi layer perceptron

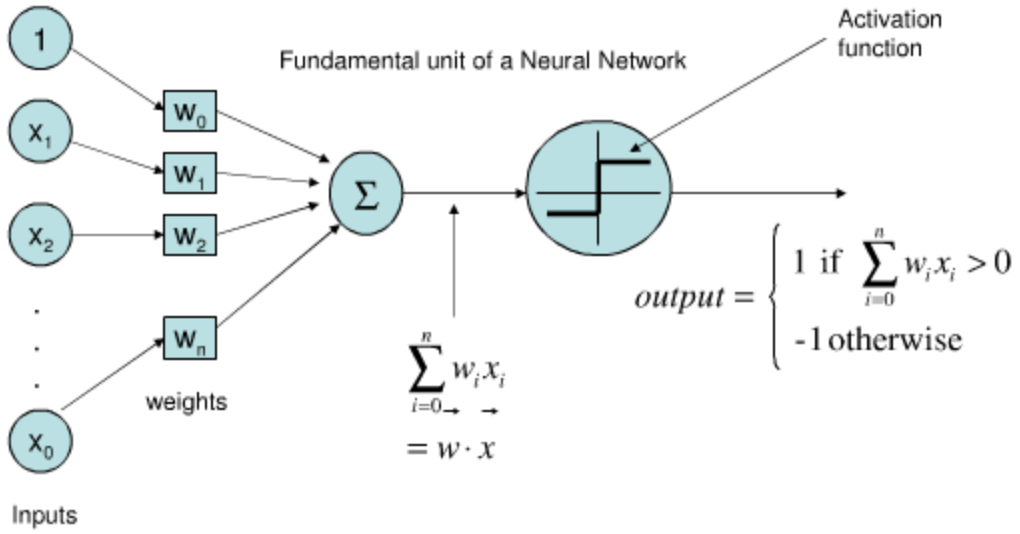
E′ usato sia per problemi di classificazione che di clustering

Il classificatore lineare , il caso binario:

Obiettivo, trovare una funzione che possa suddividere i campioni in due cluster separati.

I campioni possono essere linearmente o non linearmente separabili

$f(x) = w^T x + b$ questa è la funzione di attivazione. (inline equation per rifare)



I Pesi (Weights - $w_1, w_2, \ldots, w_n$)

I pesi determinano l'**importanza** di ogni singolo input.
Se l'input x_1 è "Piove?", questo fattore avrà un impatto enorme sulla tua decisione di andare a correre. Quindi, la rete neurale assegnerà un peso (w_1) molto alto (in

questo caso negativo, perché disincentiva l'uscita). Se x_2 è "C'è vento?", potrebbe avere un peso (w_2) inferiore.

• **MatemIl Bias (w_0 nell'immagine)aticamente:** Ogni input viene moltiplicato per il suo peso ($w_i \cdot x_i$). Un peso grande amplifica l'input; un peso vicino a zero lo ignora; un peso negativo ne inverte l'effetto.

• **Apprendimento:** Quando diciamo che una rete neurale "impara" (tramite il Gradient Descent che abbiamo visto prima), intendiamo dire proprio che **modifica questi pesi** per dare la giusta importanza a ogni dato.

Il Bias (w_0 nell'immagine)

Guarda attentamente il primo input in alto nell'immagine: è un 1 fisso, collegato al peso w_0. Questo è il trucco matematico per rappresentare il **Bias**.

A cosa serve? Il bias rappresenta la **predisposizione di base** del neurone, indipendentemente dagli input. Tornando all'esempio della corsa: se sei un maratoneta super motivato, la tua predisposizione a uscire (il tuo bias) è altissima. Usciresti anche se piovesse un po'.

• **Matematicamente:** È una costante che viene aggiunta alla somma degli input pesati. Funziona esattamente come l'intercetta q nell'equazione di una retta ($y = mx + q$).

• **Senza il bias:** La somma passerebbe sempre per lo zero (l'origine). Il bias permette alla funzione di attivazione di traslare a destra o a sinistra, rendendo la rete molto più flessibile e capace di imparare.

La funzione di attivazione:

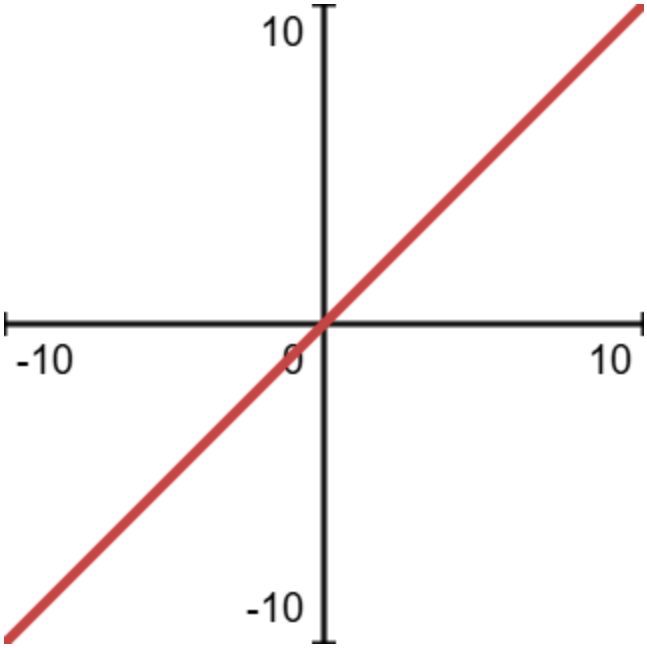
La sua funzione principale è una sola:

introdurre la non-linearità

Se non usassimo le funzioni di attivazione (o usassimo solo quelle lineari), la nostra rete neurale, non importa quanti strati abbia, si comporterebbe come un semplice modello di regressione lineare. Non sarebbe in grado di imparare pattern complessi, curve o relazioni intricate nei dati (come riconoscere un volto in una foto o capire il sarcasmo in un testo).

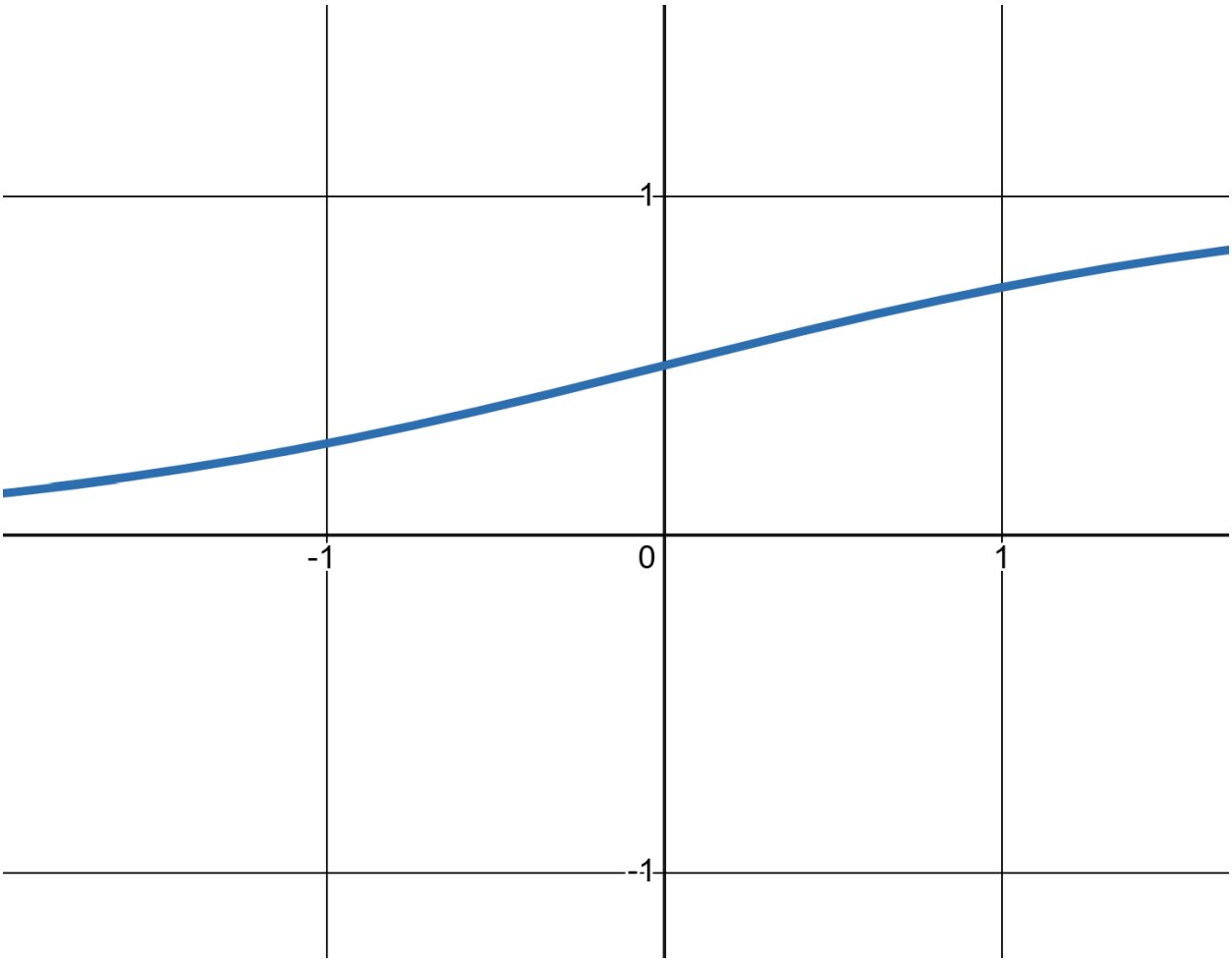
Abbiamo 4 esempi di funzioni di attivazione:

1. Linear (Lineare)
• **Come funziona:** Restituisce esattamente l'input che riceve. La formula è semplicemente $f(x) = x$
• **Range di output:** Da $-\infty$ a $+\infty$.
• **Quando si usa:** Praticamente **mai** nei livelli nascosti (hidden layers), altrimenti la rete perde la capacità di imparare cose complesse. Si usa a volte nel livello di output (ultimo strato) se stai prevedendo un valore numerico continuo, come il prezzo di una casa (Regressione).



Sigmoid (Sigmoide)

- **Come funziona:** "Schiaccia" qualsiasi numero in ingresso in un valore compreso tra 0 e 1.
- **Range di output:** $0 < x < 1$ (Corretto, come nei tuoi appunti!).
- **Formula:** $f(x) = \frac{1}{1+e^{-x}}$
- **Quando si usa:** È molto utile nell'ultimo strato se devi fare una **classificazione binaria** (es. capire se un'email è spam o non spam), perché l'output può essere letto come una probabilità (es. 0.8 = 80% di probabilità). Oggi si usa meno nei livelli intermedi perché rallenta l'apprendimento.

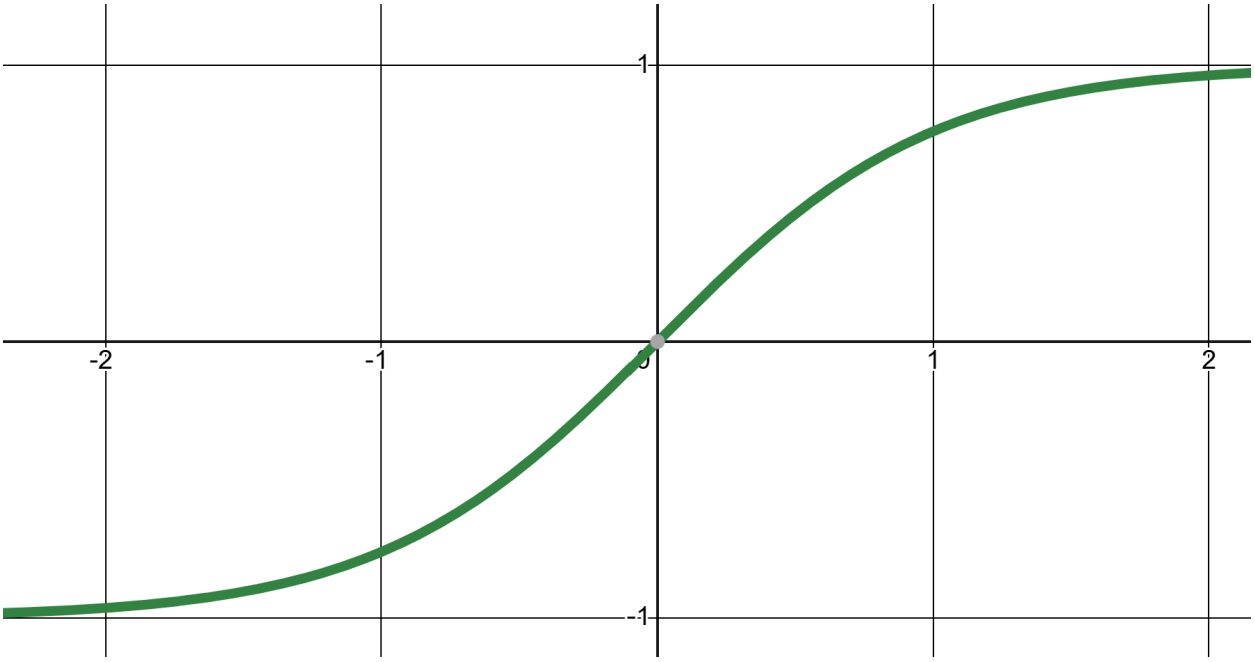


Tanh (Tangente Iperbolica)

- **Come funziona:** È molto simile alla Sigmoide, ma "schiaccia" i numeri tra -1 e 1

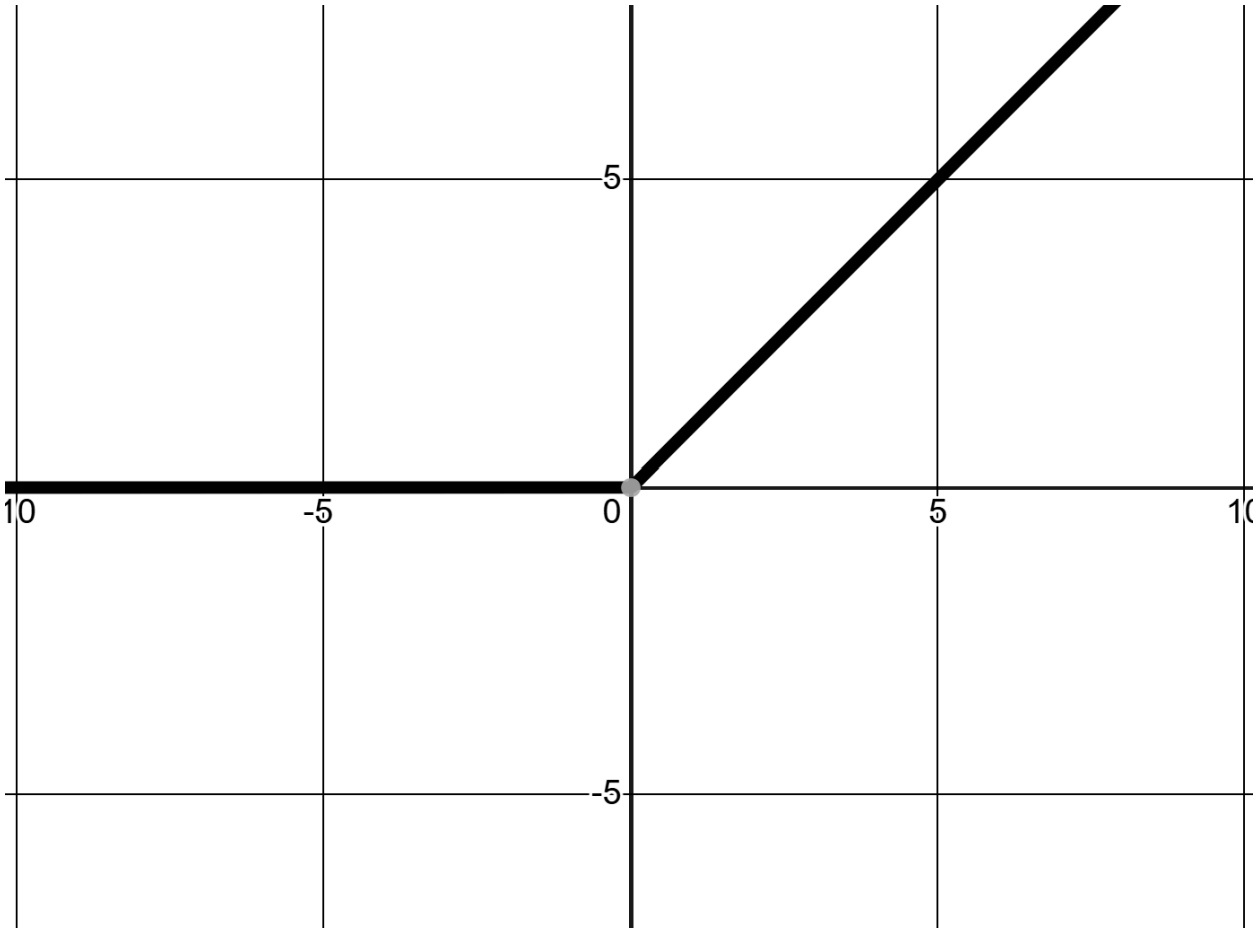
ed è centrata sullo zero.
• **Range di output:** $-1 < x < 1$ *(Questo è il range che avevi per errore associato alla ReLU).*
• **Formula:** $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
• **Quando si usa:** Generalmente preferibile alla Sigmoide nei livelli nascosti, perché il fatto di essere

centrata sullo zero aiuta l'algoritmo del Gradiente Discendente a convergere più velocemente.



ReLU (Rectified Linear Unit)
• **Come funziona:** È semplicissima ma potentissima. Se il numero in ingresso è negativo, restituisce zero. Se è positivo, restituisce il numero stesso.
• **Range di output:** Da 0 a $+\infty$
• **Formula:** $f(x) = \max(0, x)$
• **Quando si usa:** È la **regina incontrastata dei livelli nascosti**. È facilissima da

calcolare per il computer e risolve molti problemi matematici che rallentano l'apprendimento delle reti profonde.



la Funzione di Perdita (Loss Function):

$$\mathcal{L}(\cdot) = \sum_{n=1}^{N} e\left\{\mathbf{oh}_n, \mathbf{g}\left(\mathbf{b}^{(2)} + \mathbf{W}^{(2)}\left(\mathbf{f}(\mathbf{b}^{(1)} + \mathbf{W}^{(1)}\mathbf{x})\right)\right)\right\}$$

serve a **misurare l'errore**

1. La Funzione di Perdita (Loss Function): "Il Voto all'Esame"

La Loss Function calcola quanto le previsioni della tua rete neurale si discostano dalla realtà. Più il valore è alto, peggio sta performando il modello.

Questa formula descrive quello che viene chiamato **Forward Pass** (passaggio in avanti) in una rete a due strati (Multilayer Perceptron). Leggendola dall'interno verso l'esterno:

• **L'ingresso (Input):** Tutto parte dal dato di input $\mathbf{x}$

• **Lo strato nascosto (Hidden Layer):** L'input viene moltiplicato per i pesi e per il bias. Il risultato passa attraverso una funzione di attivazione $\mathbf{f}$ (che introduce non linearità, permettendo alla rete di imparare pattern complessi).

• **Lo strato di uscita (Output Layer):** Il risultato dello strato nascosto subisce lo stesso trattamento con i pesi $\mathbf{W}^{(2)}$e bias $\mathbf{b}^{(2)}$, passando poi per la funzione di attivazione finale $\mathbf{g}$ (che genera la predizione finale della rete).

• **Il calcolo dell'errore:** La funzione e prende questa predizione e la confronta con $\mathbf{oh}_n$, che è il risultato reale e corretto (il "target" formattato come vettore one-hot).

• **Il totale:** Il simbolo della sommatoria indica che questo calcolo dell'errore viene ripetuto e sommato per tutti gli N dati presenti nel tuo dataset di addestramento.

Simbolo	Descrizione
$\mathcal{L}(\cdot)$	Funzione di Perdita (Loss Function) totale
N	Numero totale di campioni nel dataset
$e\{\dots\}$	Funzione di errore tra valore reale e predetto

$\mathbf{oh}_n$	Target reale (vettore one-hot encoding)
$\mathbf{x}$	Vettore di input
$\mathbf{W}^{(1)}, \mathbf{b}^{(1)}$	Pesi e bias del primo strato (**Hidden Layer**)
$\mathbf{f}$	Funzione di attivazione del primo strato
$\mathbf{W}^{(2)}, \mathbf{b}^{(2)}$	Pesi e bias del secondo strato (**Output Layer**)
$\mathbf{g}$	Funzione di attivazione dello strato di output

Gradiente Discendente:

$$\theta[n+1] = \theta[n] - \gamma \nabla J(\theta[n])$$

Questo è il processo di **Ottimizzazione**. Immagina la funzione di errore come un paesaggio montuoso: l'obiettivo della rete è scendere a valle, verso il punto più basso possibile (l'errore minimo).

• **Il punto di partenza:** $\theta[n]$ rappresenta la posizione attuale, ovvero il valore corrente di tutti i pesi e bias della rete.

• **La bussola:** $\nabla J(\theta[n])$ è il gradiente. Misura la pendenza della "montagna" dell'errore nel punto in cui ti trovi. Puntando verso la salita massima, noi dobbiamo andare nella direzione **opposta** (ecco perché c'è il segno meno - nella formula). E' la mia Loss function calcolata precedentemente, mi indica quanto sto sbagliando e che quindi devo andare nella direzione opposta di un parametro $\theta[n]$

• **Il passo:** γ (gamma) è il *Learning Rate* (tasso di apprendimento). Decide quanto deve essere grande il passo che fai verso la discesa. Un passo troppo piccolo rende l'apprendimento lentissimo; uno troppo grande rischia di farti scavallare la valle e risalire dall'altra parte.

• **L'arrivo:** $\theta[n+1]$ è il tuo nuovo set di pesi aggiornati. Al prossimo giro, la rete sarà (si spera) un po' più brava a fare le sue predizioni.

Simbolo	Nome	Significato

$\theta[n+1]$	**Nuovo Parametro**	I pesi aggiornati dopo l'errore.
$\theta[n]$	**Parametro Corrente**	I pesi attuali prima della correzione.
γ	**Learning Rate**	Il "passo" di apprendimento (quanto velocemente impara).
∇J	**Gradiente**	La direzione verso cui l'errore aumenta (noi andiamo al contrario).
$\mathbf{W}, \mathbf{b}$	**Pesi e Bias**	I parametri interni della rete (contenuti in θ).
$\mathbf{f}, \mathbf{g}$	**Funzioni di Attivazione**	Le trasformazioni non lineari (es. ReLU, Sigmoide).

Il Ciclo di Vita (Come lavorano insieme)

In sintesi, l'addestramento (Training) di una rete neurale non è altro che un ciclo continuo tra queste due formule:

1. Usa la prima formula per fare una previsione e misurare l'errore.
2. Usa la seconda formula per correggere i parametri in base a quell'errore.
3. Ripeti migliaia di volte finché l'errore non smette di scendere.

Deep Feedforward networks:

Questa formula calcola l'errore totale della rete neurale su tutto il set di dati.

$$\mathcal{L}(\cdot) = \sum_{n=1}^{N} e\left\{\mathbf{oh}_n, \mathbf{g}\left(\mathbf{b}^{(2)} + \mathbf{W}^{(2)}\left(\mathbf{f}(\mathbf{b}^{(1)} + \mathbf{W}^{(1)}\mathbf{x})\right)\right)\right\}$$

• **L(·)**: Rappresenta la Loss Function complessiva.
• **Σ (da 1 a N)**: Indica che sommiamo l'errore per tutti gli N campioni del dataset.
• **f(...) e g(...)**: Sono le funzioni di attivazione rispettivamente dell'hidden layer e dell'output layer.
• **W e b**: Sono i pesi (Weights) e i bias dei due strati della rete.

2. Regola di Aggiornamento (Gradient Descent)

Questa è la formula base per aggiornare i parametri della rete e ridurre l'errore.

Formula da copiare: $\theta[n+1] = \theta[n] - \gamma \nabla \mathcal{L}(\theta[n])$

- **θ[n+1]**: Il nuovo valore dei parametri (pesi e bias) dopo l'aggiornamento.
- **γ (gamma)**: Il Learning Rate, ovvero quanto è grande il passo che facciamo verso il minimo dell'errore.
- **∇L**: Il gradiente della funzione di perdita, che indica la direzione di massima salita (usiamo il meno per andare in discesa).

Poiché calcolare il gradiente su tutto il dataset (M) è troppo pesante, si usa un piccolo gruppo casuale di campioni (B).
$\nabla \mathcal{L}(\theta[n]) = \sum_{b=1}^{B} \nabla \varepsilon(\mathbf{oh}_b, \mathrm{mlp}(\mathbf{x}_b, \theta))|_{\theta=\theta[n]}$
• **B**: Dimensione del mini-batch (*sottoinsieme casuale di dati*).
• **mlp(x, θ)**: Rappresenta l'uscita prodotta dalla rete neurale per l'input corrente.
• **ε (epsilon)**: L'errore calcolato sul singolo campione del batch.

Backpropagation

La Backpropagation è l'algoritmo fondamentale per addestrare le reti neurali. Il *Gradient Descent* indica **come** aggiornare i pesi per ridurre l'errore, mentre la *Backpropagation* è il metodo efficiente per **calcolare** quel gradiente—ovvero la derivata della Loss rispetto a ogni singolo peso della rete.

L'addestramento di una rete neurale avviene in cicli continui chiamati epoche. Ogni ciclo si divide in quattro fasi:

1. **Forward Pass (Propagazione in avanti):**
    - I dati di input attraversano la rete neurale, strato dopo strato.
    - Ogni neurone applica i propri pesi, bias e funzione di attivazione.
    - La rete genera un output finale (la predizione).
2. **Calcolo della Loss (Errore):**

- L'output viene confrontato con il target reale (l'etichetta corretta) usando una *Loss Function* (es. MSE o Cross-Entropy).
- Si ottiene un valore numerico che indica quanto la rete ha sbagliato.

3. **Backward Pass (La vera Backpropagation):**
   - L'errore viene propagato all'indietro attraverso la rete, dall'ultimo strato (output) fino al primo (input).
   - Si calcola quanto ogni singolo peso e bias ha contribuito all'errore totale.
4. **Aggiornamento dei Pesi:**
   - Si usa l'algoritmo del *Gradient Descent* per modificare i pesi nella direzione opposta all'errore, migliorando la rete per l'iterazione successiva.

ChainRule:

Il cuore matematico della backpropagation è la **Regola della Catena (Chain Rule)** del calcolo differenziale. Una rete neurale è essenzialmente una funzione composta (funzioni dentro altre funzioni). Per trovare come cambia l'errore rispetto a un peso specifico nel primo strato, dobbiamo moltiplicare le derivate di tutti gli strati successivi.

Formula base della Chain Rule: $\frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$

Legenda dei termini:

- $\frac{\partial \mathcal{L}}{\partial w}$: Quanto cambia la Loss Function (L) al variare del peso (w). È il nostro obiettivo finale—il gradiente.
- $\frac{\partial \mathcal{L}}{\partial a}$: Quanto cambia l'errore rispetto all'attivazione (a) del neurone.
- $\frac{\partial a}{\partial z}$: La derivata della funzione di attivazione rispetto al valore grezzo (z).
- $\frac{\partial z}{\partial w}$: Come cambia il valore grezzo (z) al variare del peso (w).

Cos'è l'Argmax?

L'operazione argmax (Argument of the Maximum) è una funzione matematica che analizza un vettore di valori (es. probabilità) e restituisce l' **indice** (la posizione) dell'elemento con il valore più alto, non il valore stesso.

Come funziona (Esempio del Vettore)

Immaginiamo che una rete neurale restituisca questo vettore di probabilità come risultato finale:
`[0.1, 0.1, 0.2, 0.6]`

Considerando che in informatica gli indici partono da 0:

- La funzione **Max** restituirebbe `0.6` (il valore maggiore).
- La funzione **Argmax** restituisce `3` (l'indice in cui si trova il valore maggiore).

Batch Size ed Epoche (Training Deep Learning)

> **Contesto:**
> Quando addestriamo un LLM (Large Language Model), abbiamo a disposizione un dataset immenso (miliardi di parole/token). Il modello non può "leggere" tutto il dataset in un colpo solo, né aggiornare i suoi pesi riga per riga. Deve farlo a blocchi e attraverso cicli ripetuti. Qui entrano in gioco la **Batch Size** e le **Epoche**.

Le Epoche (Epochs)

Un'epoca rappresenta **un passaggio completo della rete neurale su tutto il dataset di addestramento**.

- **L'Analogia:** Immagina di dover preparare un esame su un libro di 1000 pagine. Leggere il libro intero dalla prima all'ultima pagina equivale a 1 Epoca.
- **Come funziona:** Se imposti `Epochs = 50`, significa che la rete neurale analizzerà l'intero dataset per 50 volte durante il suo addestramento.

Perché è importante definirle correttamente?
Il numero di epoche serve a bilanciare l'apprendimento ed evitare due dei problemi più comuni nel Machine Learning:

1. **Underfitting (Poche epoche):** La rete ha visto i dati troppo poche volte e non ha ancora colto i pattern fondamentali (es. non sa ancora distinguere bene le forme di base). Le sue previsioni sono molto imprecise.
2. **Overfitting (Troppe epoche):** La rete ha visto gli stessi dati così tante volte che li ha **imparati a memoria**, compreso il "rumore" o i difetti specifici di quelle foto. Sarà perfetta sui dati di addestramento, ma fallirà miseramente su immagini nuove mai viste prima.

2. Dimensione del Batch (Batch Size)

La Batch Size indica **il numero di campioni (es. immagini o righe di dati) che la rete elabora contemporaneamente** prima di calcolare l'errore e aggiornare i suoi parametri (pesi e bias).

- **L'Analogia:** Invece di leggere tutto il libro prima di capire se hai imparato qualcosa (Full Batch), leggi un capitolo di 32 pagine alla volta (Batch Size = 32), fai un riassunto mentale per correggere i tuoi errori, e poi passi alle 32 pagine successive.

Perché è importante definirla correttamente?

1. **Limiti Hardware (Memoria):** È il limite fisico principale. Se hai una scheda video (GPU) con 8GB di memoria e provi a caricare un batch di 10.000 immagini in alta risoluzione contemporaneamente, il programma andrà in crash (Errore *Out Of Memory - OOM*).
2. **Qualità dell'apprendimento (Stabilità vs Esplorazione):**
    - **Batch Size piccola (es. 16, 32):** La rete aggiorna i pesi molto spesso. Il percorso verso l'errore minimo sarà "frastagliato" e rumoroso, ma questo rumore spesso aiuta la rete a non bloccarsi in soluzioni sub-ottimali (minimi locali).

- **Batch Size grande (es. 256, 512):** La direzione in cui i pesi vengono aggiornati è molto più precisa e stabile, e i calcoli sono più parallelizzabili (veloci su GPU), ma richiede molta più memoria.

Come interagiscono Epoche e Batch Size?

Per capire la relazione tra Dataset, Epoche, Batch Size e le "Iterazioni" (o *Steps*, cioè il numero effettivo di volte in cui i pesi vengono aggiornati in un'epoca), si usa questa formula:

$$Iterazioni\ per\ singola\ Epoca = (Dimensione\ Totale\ del\ Dataset) / (Batch\ Size)$$

Esempio Pratico: Se hai 1000 immagini e una Batch Size di 100, ci vorranno 10 iterazioni per completare 1 Epoca.

Parametro	Cosa rappresenta in pratica?	Rischio se è troppo ALTO	Rischio se è troppo BASSO
Epoche	Quante volte studio l'intero set di dati	Overfitting (impara a memoria)	Underfitting (non impara i pattern)
Batch Size	Quanti dati processo ad ogni aggiornamento	Crash di memoria (OOM) sulla GPU	L'addestramento diventa troppo lento

Attenzione!:

La "Batch Size" non significa prendere un piccolo gruppo di dati e ignorare il resto. Significa **dividere l'intero dataset in "pacchetti" da processare uno alla volta**.

🔀 **Batch, Epoche e il Mescolamento dei Dati (Shuffling)Risposta Veloce:** > Quando imposti una Batch Size di 100 su un dataset di 1000 immagini, **NON** addestri la rete sempre sulle stesse 100 immagini. Invece, dividi le tue 1000 immagini in **10 pacchetti diversi da 100 immagini ciascuno**.

⚙️ **Come funziona esattamente (L'esempio delle 1000 immagini)**
Immaginiamo di avere 1000 immagini, una Batch Size di 100 e di voler fare 3

Epoche.

All'inizio dell'epoca, la rete non ha ancora visto nulla.

Epoca 1

1. **Iterazione 1:** Prende le immagini da 1 a 100 $\rightarrow$ calcola l'errore $\rightarrow$ aggiorna i pesi.
2. **Iterazione 2:** Prende le immagini da 101 a 200 $\rightarrow$ calcola l'errore $\rightarrow$ aggiorna i pesi.
3. *...continua così...*
4. **Iterazione 10:** Prende le immagini da 901 a 1000 $\rightarrow$ calcola l'errore $\rightarrow$ aggiorna i pesi.

F**ine Epoca 1!** La rete ha visto **tutte e 1000** le immagini, ma elaborandole a gruppi di 100.
Epoca 2 (Il potere dello Shuffling).

Shuffle

Prima di iniziare la seconda epoca, succede una cosa fondamentale: **il dataset viene mescolato (Shuffled).**
Le 1000 immagini vengono rimescolate come un mazzo di carte.

Epoca 2

1. **Iterazione 1 (Nuova):** Prende le prime 100 immagini del nuovo mazzo mescolato (es. le immagini 4, 87, 912, 23...).
2. **Iterazione 2 (Nuova):** Prende le successive 100 immagini.
3. *...fino alla fine del mazzo...*

🎉 **Fine Epoca 2!** La rete ha rivisto tutte le 1000 immagini per la seconda volta, ma aggregate in **gruppi diversi** rispetto all'Epoca 1.

Epoca 3 e successive:

Si mescola di nuovo il mazzo da 1000 e si ripete il processo.

Ritorniamo alla formula precedente, sapendo che ho una dimensione totale di 1000 e ogni sotto-campione è 100 dovrò fare 10 iterazioni per concludere un epoca.

$$Iterazioni\ per\ singola\ Epoca = (Dimensione\ Totale\ del\ Dataset)\ /\ (Batch\ Size)$$

Se non mescolassimo i dati a ogni epoca e dessimo alla rete i pacchetti sempre nello stesso identico ordine, la rete potrebbe imparare a memoria la *sequenza* delle immagini invece delle caratteristiche reali (es. "Dopo 100 gatti arrivano sicuramente 100 cani, quindi rispondo cane").

Mescolando i batch, introduciamo una sana dose di "caos" che costringe la rete a generalizzare e a concentrarsi su *cosa* c'è nell'immagine, non su *quando* le viene mostrata.

Operazioni di quantizzazione e pruning per alleggerire i modelli:

Le reti neurali addestrate sono pesanti, lente e consumano molta energia (batteria/elettricità). Per portarle nel mondo reale (es. su smartphone, smartwatch o server efficienti), dobbiamo ridurne le dimensioni e velocizzarne i calcoli senza perdere troppa "intelligenza" (accuratezza).

Il **Pruning** consiste nell'eliminare fisicamente le connessioni (i pesi) o interi neuroni che non contribuiscono in modo significativo al risultato finale della rete.

- **L'Analogia:** Immagina un albero con tantissimi rami. Molti di questi rami sono secchi o non producono foglie. Tagliandoli (potatura), l'albero diventa più leggero e compatto, ma continua a vivere e a fare ombra esattamente come prima.
- **Come funziona in pratica:** Dopo l'addestramento, si analizzano i pesi della rete. Molti pesi avranno valori vicinissimi allo zero (es. `0.0001`). Significa che quell'input è quasi ininfluente. Il pruning prende questi pesi e li forza a `0` esatto, rimuovendo di fatto la connessione.

Tipologie di Pruning:

- **Unstructured Pruning (Non strutturato):** Si eliminano singoli pesi sparsi per la rete. *Pro:* Mantiene alta l'accuratezza. *Contro:* Crea matrici "sparse" (piene di zeri) che l'hardware standard fa fatica ad accelerare realmente.
- **Structured Pruning (Strutturato):** Si eliminano interi neuroni, filtri o canali. *Pro:* Riduce fisicamente la dimensione delle matrici, velocizzando enormemente i calcoli su qualsiasi hardware. *Contro:* È più aggressivo e rischia di abbassare l'accuratezza del modello.

📉 2. Quantizzazione (Quantization)

La **Quantizzazione** non elimina nulla, ma riduce la "risoluzione" matematica con cui vengono salvati i pesi e le attivazioni della rete.

- **L'Analogia:** Immagina di avere una foto in 4K ad altissima risoluzione (Float32). Se la comprimi a 1080p (Int8), l'immagine pesa molto meno sul disco e si carica più in fretta. Certo, se fai uno zoom estremo perdi dei dettagli, ma a occhio nudo continui a distinguere perfettamente un cane da un gatto.
- **Come funziona in pratica:** Di default, il Deep Learning usa numeri decimali a 32 bit (`Float32`, es. `3.14159265...`). La quantizzazione arrotonda questi numeri a formati più piccoli, come interi a 8 bit (`Int8`, es. `3`).

I Vantaggi della Quantizzazione:

1. **Meno Memoria (RAM/VRAM):** Passare da 32-bit a 8-bit riduce il peso del modello di ben 4 volte (es. un modello da 4GB diventa da 1GB).
2. **Maggiore Velocità:** I processori (CPU/GPU/NPU) eseguono operazioni matematiche con numeri interi (Int8) molto più velocemente rispetto ai numeri decimali complessi (Float32).
3. **Meno Energia:** Richiede meno potenza elettrica, fondamentale per i dispositivi mobili.

Tipologie di Quantizzazione:

- **PTQ (Post-Training Quantization):** Addestri la rete normalmente in Float32, e alla fine converti tutto in Int8. È veloce, ma a volte il modello perde troppa accuratezza a causa degli arrotondamenti.
- **QAT (Quantization-Aware Training):** Durante l'addestramento, il modello "simula" la quantizzazione. In questo modo impara fin da subito a convivere con la minore precisione, ottenendo risultati finali molto superiori.

Tecnica	Cosa fa ai parametri?	Vantaggio Principale	Impatto sull'Hardware
Pruning	Ne **riduce il numero** eliminando i meno importanti.	Riduce la complessità architetturale.	Richiede hardware specifico se non è "strutturato".
Quantizzazione	Ne **riduce la precisione** (da 32-bit a 8-bit o meno).	Riduce drasticamente il peso in MB/GB.	Rende l'inferenza molto più veloce su quasi tutti i chip.